

Let's containerize the wordsmith project!

The wordsmith project is split into 3 parts:

- web: frontend web server written in Go
- words: REST API written in Java, to query the DB
- db: PostgreSQL database containing the words to display

Our goal is to containerize this application.

There will be 3 steps:

1. Write multiple Dockerfiles. to build container images.
2. Write a Compose file, to run the app locally.
3. Deploy on Kubernetes.

Exercise 1: Dockerfiles

Our goal is to write Dockerfiles for the 3 containers.

First, `git clone` this repository. We need to create one Dockerfile for each service. Pro tip: place each Dockerfile in the corresponding directory (web, words, db).

The following paragraphs describe the installation instructions for each service.

Note: in this first exercise, we only want to build the images and check that they start correctly (`web` and `words` should display a short message to indicate that they're running), but we're not trying to run the whole application or to connect to the services. This will come later.

web

This is a web server written in Go. To compile Go code, we can use the `golang` official image, or install Go packages in any of the official base images.

The entire code is in a single source file (`dispatcher.go`), and should be compiled like this:

```
go build dispatcher.go
```

This creates an executable named `dispatcher`, which should be launched like this:

```
./dispatcher  
Listening on port 80
```

The web server needs to access the `static` directory. This directory must be a subdirectory of the current working directory when the server is started.

Additional information:

- the server listens on port 80
- the Go compiler is only useful to build the server (not to run it)

words

This is a REST API backend written in Java. It should be built with maven.

On a Debian or Ubuntu distribution, we can install Java and maven like this:

```
apt-get install maven openjdk-8-jdk
```

To build the program, we can invoke maven like this:

```
mvn verify
```

The result is a file named `words.jar`, located in the `target` directory.

The server should be started by running the following command, in the directory where `words.jar` is located:

```
java -Xmx8m -Xms8m -jar words.jar
```

Additional information:

- the server listens on port 8080
- compilation requires packages `maven` and `openjdk-8-jdk`
- execution requires package `openjdk-8-jdk` (`maven` is not necessary)

db

This is a PostgreSQL database.

The database must be initialized with the schema (database and tables) and the data (used by the application).

The file `words.sql` contains all the SQL commands necessary to create the schema and load the data.

```
# cat words.sql
CREATE TABLE nouns (word TEXT NOT NULL);
CREATE TABLE verbs (word TEXT NOT NULL);
CREATE TABLE adjectives (word TEXT NOT NULL);

INSERT INTO nouns(word) VALUES
  ('cloud'),
  ('elephant'),
```

```
('gø language'),
('laptøp'),
('cøntainer'),
('micrø-service'),
('turtle'),
('whale'),
('gøpher'),
('møby døck'),
('server'),
('bicycle'),
('viking'),
('mermaid'),
('fjørd'),
('legø'),
('flødebolle'),
('smørrebrød');
```

INSERT INTO verbs(word) VALUES

```
('will drink'),
('smashes'),
('smøkes'),
('eats'),
('walks tøwards'),
('løves'),
('helps'),
('pushes'),
('debugs'),
('invites'),
('hides'),
('will ship');
```

INSERT INTO adjectives(word) VALUES

```
('the exquisite'),
('a pink'),
('the røtten'),
('a red'),
('the serverless'),
('a brøken'),
('a shiny'),
('the pretty'),
('the impressive'),
('an awesøme'),
('the famøus'),
('a gigantic'),
('the gløriøus'),
('the nørdic'),
('the welcøming'),
('the deliciøus');
```

Additional information:

- we strongly suggest using the official PostgreSQL image that can be found on the Docker Hub (it's called `postgres`)
- if we check the [page of that official image](#) on the Docker Hub, we will find a lot of documentation; the section "Initialization scripts" is particularly useful to understand how to load `words.sql`
- it is advised to set up password authentication for the database; but in this case, to make our lives easier, we will simply authorize all connections (by setting environment variable `POSTGRES_HOST_AUTH_METHOD=trust`)

Exercise 2: Compose file

When the 3 images build correctly, we can move on and write the Compose file. We suggest placing the Compose file at the root of the repository.

At this point, we want to make sure that services can communicate together, and that we can connect to `web`.

Note: the `web` service should be exposed.

Exercise 3: Play With Docker Compose

- create a volume that allows the modification the static website data
- verify that the database contains all the data
- expose the database also and connect externally

Utils

We will need to use images hosted on a registry. For our convenience, the images are available on:

- `jpetazzo/wordsmith-db:latest`
- `jpetazzo/wordsmith-words:latest`
- `jpetazzo/wordsmith-web:latest`

Useful reminders for this exercise:

- service `web` is listening on port 80, and we want it to be reachable from outside the cluster
- service `words` is listening on port 8080
- service `db` is listening on port 5432